

Final Exam Topics

14. Basic algorithms

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

Basic algorithms and data structures

Asymptotic behavior of functions, efficiency of algorithms. Comparison sorts (insertion, merge, quick, and heap sort), lower bound on the number of operations. Sorting in linear time (bucket, counting, and radix sort). Data compression (naive, Huffman, LZW). String matching (brute-force, quicksearch, KMP).

1 Asymptotic behavior of functions, efficiency of algorithms

TODO

2 Comparison sorting algorithms (bubble and insertion sort, and tournament, heap, quick, and merge sort)

Bubble and insertion sort are classic, with n^2 operation requirement; the others are efficient, with $n \log(n)$ time.

2.1 Bubble sort

It bubbles the largest value to the end with swaps; this we drop at the end of every cycle. It is not used in practice.

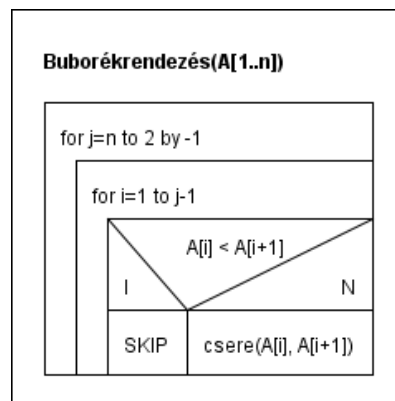


Figure 1: Bubble sort

2.2 Insertion sort

For small n (about 30) this sort is the best.

Here moving an element is always 1 assignment (in bubble sort a swap is 3 assignments). It can also be implemented

on a list; in this case we adjust the pointers and the elements stay in place.

$A[1..j]$ is sorted, $j = 1..n$.

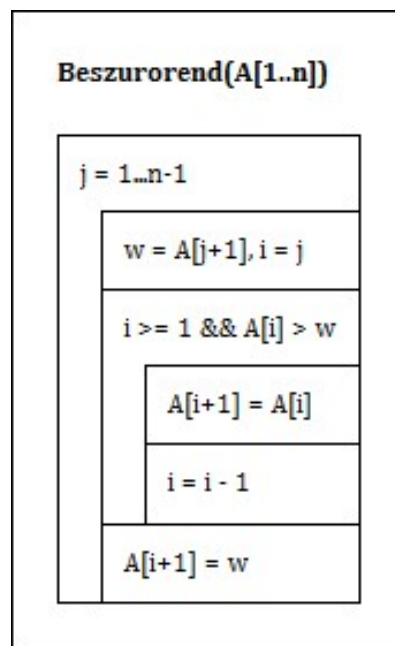


Figure 2: Insertion sort

2.3 Tournament sort

It is not used in practice.

Its basis is a complete binary tree, a tournament tree. We represent it level by level in an array.

1. Filling the tournament tree (playing the tournament). The maximum is in the root, writing it to the output.
2. $(n - 1)$ times
 - a) finding the place of the maximal element appearing in the root at the leaf level and writing $-\infty$ in its place
 - b) we replay the whole thing (the branch where it was) $\rightarrow$ the 2nd best gets up to the root

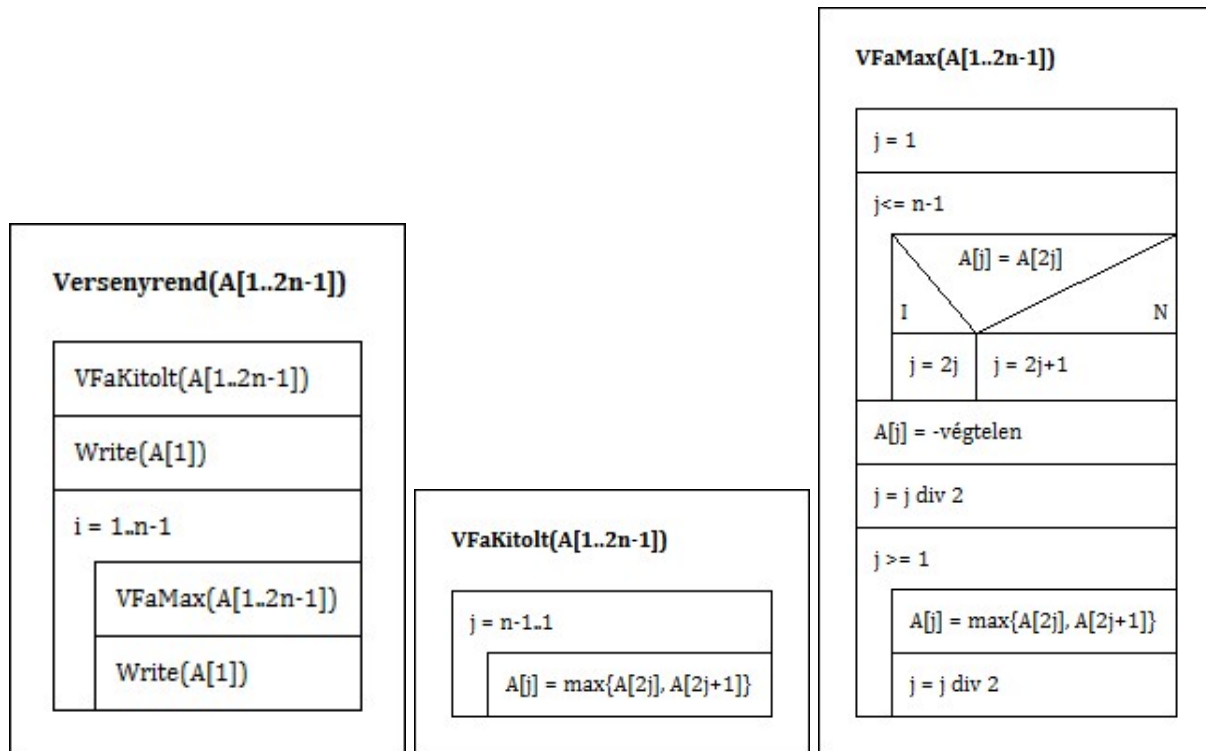


Figure 3: Tournament sort

2.4 Heap sort

1. Building the initial heap. From an unsorted input array we make a content invariant that already has a heap structure. Principle: with swaps we sink the element toward the larger child if it is smaller than the larger child. The sinking can reach the vertex that has no right child.
2. $(n - 1)$ times
 - a) swapping the root element and the rightmost (=last) active element of the bottom level, and making the element that got down after the swap inactive
 - b) sinking the element that got into the root on the active heap

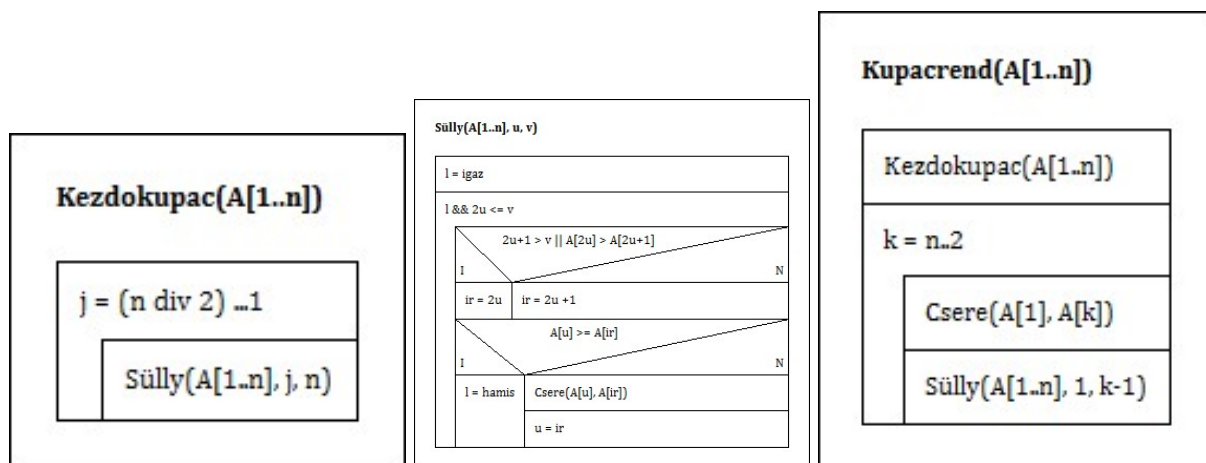


Figure 4: Heap sort

When building the initial heap, and during the cycle, the manner of sinking differs slightly, since in the first case the changing element sinks on the whole heap, in the second the root sinks on the active heap. The algorithm shown in the figure performs both operations.

2.5 Quick sort

Its principle: we randomly choose an element. We put the elements smaller than it to its left, the larger ones to its right, and put the element between the two parts. A recursive algorithm.

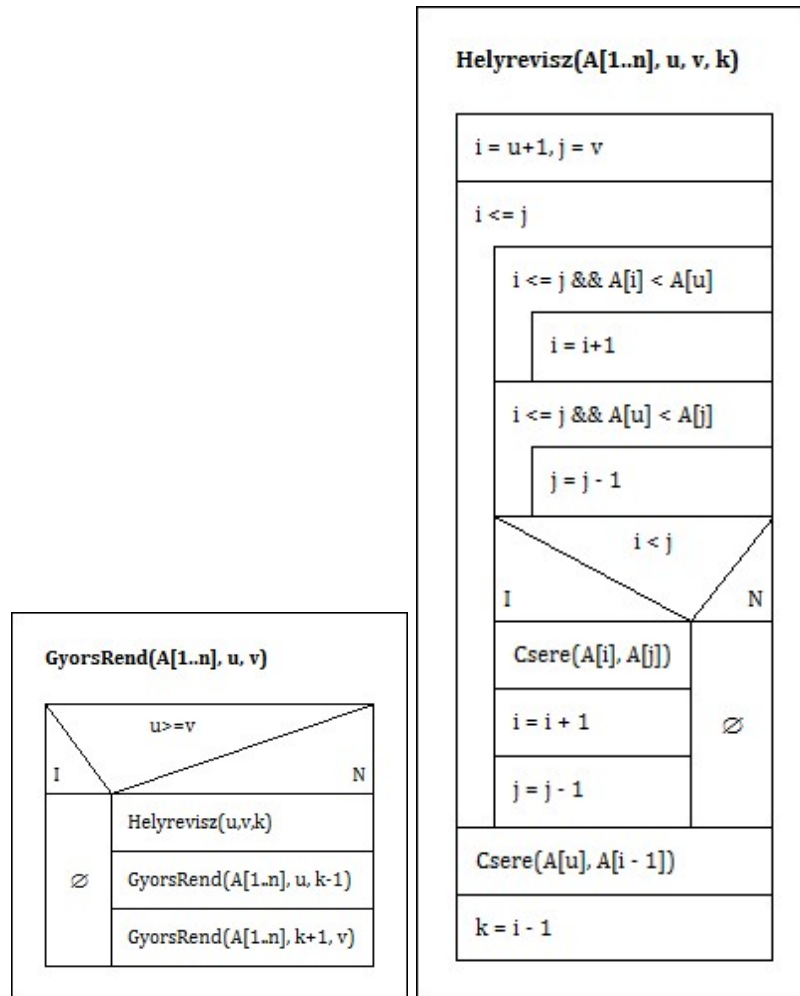


Figure 5: Quick sort

2.6 Merge sort

Its basis: merging 2 sorted sequences. We can apply this top-down (recursive) or bottom-up (iterative), the latter for sequential files.

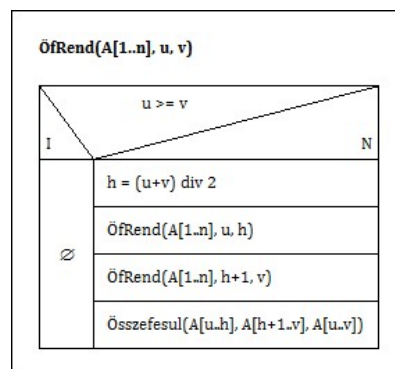


Figure 6: Merge sort

3 The lower bound on the number of operations for comparison sorts

3.1 Operation requirement

We designate the dominant operations, and look at how many times they are executed as a function of the input size n . The general notation is $T(n)$, but it can be concrete too, e.g. $Cs(n)$ [swap]. $mT(n)$ is the minimal operation requirement, $MT(n)$ the maximal, and $AT(n)$ the average.

Θ : of the same order of magnitude, can be squeezed between two constants

$\mathcal{O}$: order-of-magnitude upper estimate, o : equality is not allowed

Ω : order-of-magnitude lower estimate, ω : equality is not allowed

3.2 Lower bound

For example: maximum selection of n elements requires at least $(n-1)$ comparisons. Its proof: If there were fewer comparisons than this, then at least 1 element was left out, and with this we can reach a contradiction.

Decision tree: An algorithm for input of size n . The cycles straighten into a finite-length chain, and the trace of the execution gives a tree structure. Perfect tree: every internal point has 2 children. There is no better algorithm than this, because $2^{h(t)} \geq n!$, in the case of a comparison sort, $n!$ inputs.

3.3 Lower bound in the worst case

Theorem: $MO_R(n) = \Omega(n \log n)$ For the most unfavorable permutation at least $n \log n$ comparisons. Proof: $\log_2 n! \leq n \log_2 n = \Omega(n \log n)$, and $MO_R(n) = h(t) \geq \log_2 n!$ (because of the lemma) $\Rightarrow MO_R(n) = \Omega(n \log n)$.

3.4 Lower bound in the average case

Let every input be equally probable ($\frac{1}{n!}$).

$AO_R(n) = \frac{1}{n!} \sum_{p \in \text{Perm}(n)} O_R(p)$, and it is easy to see that $\sum_p O_R(p) = l\text{hsum}(h(t_R(n)))$ [leaf-height-sum].

Lemma: Among the perfect trees containing $n!$ leaves, the value $l\text{hsum}(h(t_R(n)))$ is smallest for those that are almost complete.

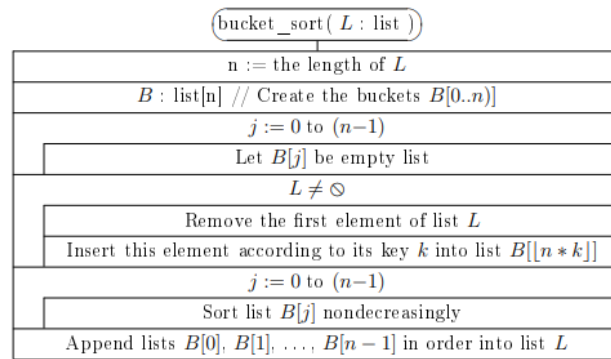
Theorem: $AO_R(n) = \Omega(n \log n)$.

4 Sorting in linear time (bucket, counting, and radix sort)

4.1 Bucket sort

Bucket sort is a sorting algorithm generally used for sorting data sets that work with uniformly distributed values.

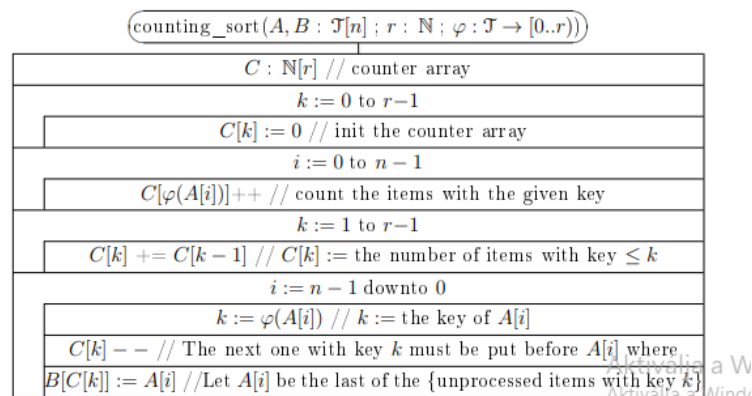
- In the input data set we determine a range or interval in which the distribution of the values is nearly the same. This range is usually the interval between the minimal and maximal value.
- We create one or more “buckets” that are located within the range. The number of buckets can be constant or variable, and depends on how we divide the range.
- We place the data into the appropriate buckets based on their key. This can happen, for example, by computing the integer part of the key value or with the help of a hash function.
- We sort the data set in every bucket. This sorting step can be the application of any other sorting algorithm, for example insertion sort or quick sort.
- We merge the sorted data sets originating from the individual buckets, so that the sorted output data set is produced.



4.2 Counting sort

Counting sort is an efficient and stable (preserving the order of elements with equal value) sorting algorithm with linear time complexity.

- First we determine the largest and smallest value of the input data set, in order to determine the range over which the values are distributed. This step is necessary so that we can create a counting array that contains the number of occurrences of the different values.
- We create a counting array whose size coincides with the size of the range. Every element of the counting array is initially 0.
- We go through the input data set, and count the occurrence of every element in the counting array. We identify the elements by the index of the counting array.
- In the counting array we update the occurrence counts of the elements by adding the occurrence count of the previous element. This step helps us determine the final place of the given element in the sorted data set.
- We go through the input data set again, and insert every element into the appropriate position of the sorted data set based on the counting array. Meanwhile we decrease the occurrence count of the given element in the counting array, in order to handle possible equal values.
- The resulting sorted data set is the sorted version of the input data set.



4.3 Radix sort

Radix sort is an efficient, stable, and non-comparison sorting algorithm generally applied for sorting integers.

- First we determine the largest value of the input data set. This is necessary in order to determine how many digits have to be taken into account during the sorting.

- We begin the sorting from the smallest digit (least significant) to the largest (most significant). This can be the units digit, tens digit, hundreds digit, etc.
- We sort the input data set by the current digit. This usually happens by applying a stability-preserving sorting algorithm, for example insertion sort or counting sort.
- We rearrange the sorted data set and place it in the resulting order. This step ensures the stability, so that the order of elements with equal value does not change.
- We repeat steps 2-4 for all digits, from least significant to most significant. Thereby the sorting in the data set happens by every digit.
- The resulting sorted data set is the sorted version of the input data set.

Az input (azaz bemeneti) lista, szimbolikus jelöléssel ($r = 4; d = 3$):

$L = \langle 103, 232, 111, 013, 211, 002, 012 \rangle$

1. menet (a számok jobbról 1., azaz jobbszélső számjegyei szerint):

$B_0 = \langle \rangle$

$B_1 = \langle 111, 211 \rangle$

$B_2 = \langle 232, 002, 012 \rangle$

$B_3 = \langle 103, 013 \rangle$

$L = \langle 111, 211, 232, 002, 012, 103, 013 \rangle$

2. menet (a számok jobbról 2., azaz középső számjegyei szerint):

$B_0 = \langle 002, 103 \rangle$

$B_1 = \langle 111, 211, 012, 013 \rangle$

$B_2 = \langle \rangle$

$B_3 = \langle 232 \rangle$

$L = \langle 002, 103, 111, 211, 012, 013, 232 \rangle$

3. menet (a számok jobbról 3., azaz balszélső számjegyei szerint):

$B_0 = \langle 002, 012, 013 \rangle$

$B_1 = \langle 103, 111 \rangle$

$B_2 = \langle 211, 232 \rangle$

$B_3 = \langle \rangle$

$L = \langle 002, 012, 013, 103, 111, 211, 232 \rangle$

Figure 7: Example, since it is more understandable than the algorithm

5 Data compression

5.1 Naive data compression

We code the text to be compressed character by character, with fixed-length bit sequences. $\Sigma = \sigma_1, \sigma_2, \dots, \sigma_i$ is the alphabet.

Each character can be coded with $\lceil \lg d \rceil$ bits, since on $\lceil \lg d \rceil$ bits $2^{\lceil \lg d \rceil}$ different binary codes can be represented, and $2^{\lceil \lg d \rceil} \geq d > 2^{\lceil \lg d \rceil - 1}$, that is, on $\lceil \lg d \rceil$ bits d different codes can be represented, but on one fewer bit no longer. In: Σ^* is the text to be compressed. With the notation $n = |\text{In}|$ it can be coded with $n * \lceil \lg d \rceil$ bits. E.g. for the text ABRAKADABRA $d = 5$ and $n = 11$, from which the length of the compressed code is $11 * \lceil \lg 5 \rceil = 11 * 3 = 33$ bits. (Of the 3-bit codes any 5 can be assigned to the 5 letters.) The compressed file also contains the code table. The code table of the above ABRAKADABRA text can be, e.g., the following:

karakter	kód
<i>A</i>	000
<i>B</i>	001
<i>D</i>	010
<i>K</i>	011
<i>R</i>	100

5.2 Huffman algorithm

The essence of compression with the Huffman algorithm is that we code the more frequently occurring elements (characters) with shorter, and the less frequently occurring ones with longer code words.

For this we have to be aware of the frequency (or relative frequency) of the individual characters. Based on these we build a so-called Huffman tree, in which we label the edges with the letters of the code, the letters to be coded are located on the leaves of the tree, and based on the labels of the path leading from the root to the leaves we read off the code words.

The algorithm (specifically for a binary Huffman tree):

1. We sort the symbols to be coded based on their frequencies.
2. We perform the following reduction steps until one group remains.
3. We choose the last two elements (rarest), merge them into a new group, and the frequency of this group will be the sum of the frequencies.
4. We put the group back into the sorted sequence (sorted by frequency).
5. From the group we form a new vertex, which vertex will be the parent of the two elements forming it.

Example:

Let the following 5 letters occur with the given frequencies:

A	B	C	D	E
5	4	3	2	1

Then the reduction steps are the following:

- | | | | |
|---|---|---|------|
| A | B | C | D, E |
| 5 | 4 | 3 | 3 |
- | | | |
|---------|---|---|
| C, D, E | A | B |
| 6 | 5 | 4 |
- | | |
|------|---------|
| A, B | C, D, E |
| 9 | 6 |
- | |
|---------------|
| A, B, C, D, E |
| 15 |

The Huffman tree can be seen in Figure 8.

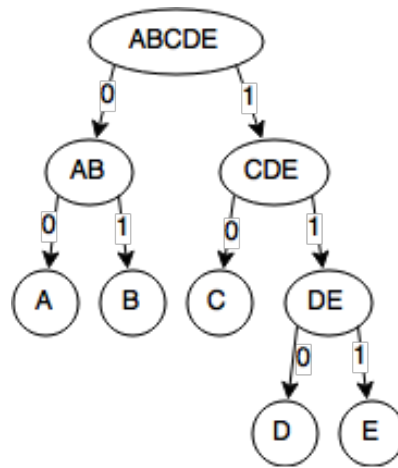


Figure 8: Huffman tree example

So the code words:

A	B	C	D	E
00	01	10	110	111

5.3 LZW algorithm

The essence of LZW (Lempel-Ziv-Welch) compression is that we continuously extend a dictionary, and assign dictionary indices to the individual words to be coded.

Coding

The coding algorithm consists of the following steps:

1. We initialize the dictionary with all words of length 1
2. We look up in the dictionary the longest string W matching the current input
3. We output the dictionary index of W , and remove W from the input
4. We add to the dictionary the concatenation of the word W and the next symbol of the input
5. We repeat from step 2.

Decoding

During decoding we also have to build the dictionary. This, however, we can do only with the help of the decoded text (part), since the next element of the dictionary is composed of the decoded word of a received code and the first character of the word after it.

So the steps of decoding:

1. We look up the word belonging to the received code in the dictionary (u), and put it to the output
2. We look up the next word (v) in the dictionary, and put it into the dictionary with the next index by concatenating its first symbol to u .
3. If there is no next word anymore, we decode but do not write into the dictionary.

It can happen that we nevertheless receive a code word that is not yet in the dictionary. This can occur if, at coding, the word currently written into the dictionary follows.

Example:

Text: AAA

Dictionary: A - 1

Then at coding we take the first character, its index in the dictionary is 1, we send this to the output. The next character is A, so we write AA into the dictionary with index 2. We delete the first character from the input. We read until we find a match in the dictionary, so we read AA (which we just put in), its index is 2, so this we send to the output. We delete AA from the input, and with this we are done. The output: 1,2

Let us decode the input 1,2! Currently in the dictionary there is only A with index 1. Let us take the first character of the input, the 1; its counterpart in the dictionary is A. Let us put this to the output. The next index is 2, but no such entry yet appears in the dictionary.

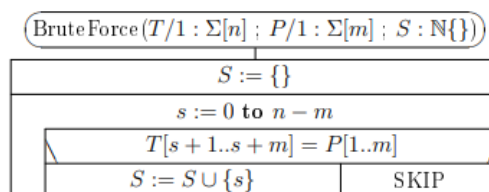
In this case, at decoding, we apply a trick. At the moment of writing into the dictionary the last character of the word to be written is not yet known (in the example we found A, but there was no entry 2). Then we write ? in place of the last character of the word to be written into the dictionary. (So A? - 2 goes into the dictionary). But now the first letter of the new entry can be known (A? - 2 is the new entry, its first letter is A). We replace the ? with this letter. (So AA - 2 will be in the dictionary).

6 String matching

6.1 Brute-force string matching

The brute force algorithm is a simple but not always efficient method applied for solving problems. During the brute force approach the algorithm tries out every possible possibility for solving the problem, and then finds the correct result.

- We determine the possible solution area of the problem. This can be a given data set, a specified interval, or values determined based on some other condition.
- We generate all possible combinations or possibilities in the solution area. This can be, for example, an iteration that goes through every possible value or combination.
- For every possible combination or possibility we check whether the given solution satisfies the conditions or criteria of the problem.
- We store the correct solutions, or choose the best result based on the problem.



6.2 Knuth-Morris-Pratt algorithm

The advantage of the Knuth-Morris-Pratt procedure compared to the Brute-Force method (compare, shift by one, etc.) is that in certain cases, if there are repeating elements in the pattern, then at a shift we can jump even several characters.

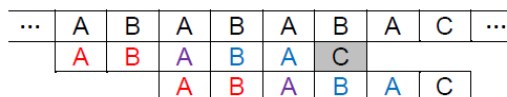


Figure 9: The KMP algorithm in the case of a multi-character shift

We determine the jump as follows: At the beginning (prefix) and the end (suffix) of the matching pattern part examined so far, we look for a character sequence that coincides. If we find such, then we can shift the pattern by as much as makes the part at its beginning fit onto the part at its end.

How much we can shift in certain cases does not have to be examined at every breakdown. If we run a modified version of the algorithm (Figure 10) on the pattern itself, we can fill an array based on which we will perform the shifts.

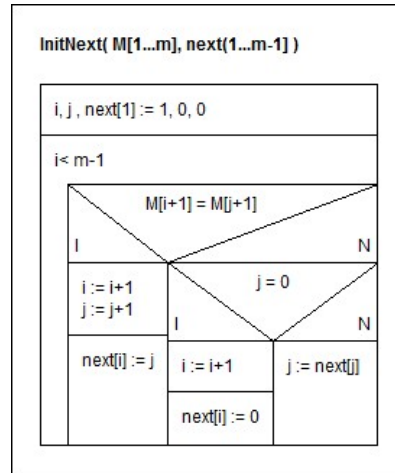


Figure 10: Filling the array regulating the KMP shifts

The algorithm (see Figure 11):

- We run two indices i and j on the text and on the pattern, respectively.
- If the $(i + 1)$ -th and $(j + 1)$ -th characters coincide, then we advance both.
- If they do not coincide, then:
 - If we were examining the first element of the pattern, then we shift the pattern by one, in other words the index of the pattern stays on the first letter, and we increase the index in the text by one ($i = i + 1$)
 - If we were not examining the first element of the pattern, then we shift by as much as is allowed. This means that we only place the index on the pattern to a smaller place ($j = next[j]$)
- We go until we reach either the end of the pattern or the end of the text. If we reached the end of the pattern, then we found the pattern in the text; if we reached the end of the text, then we did not.

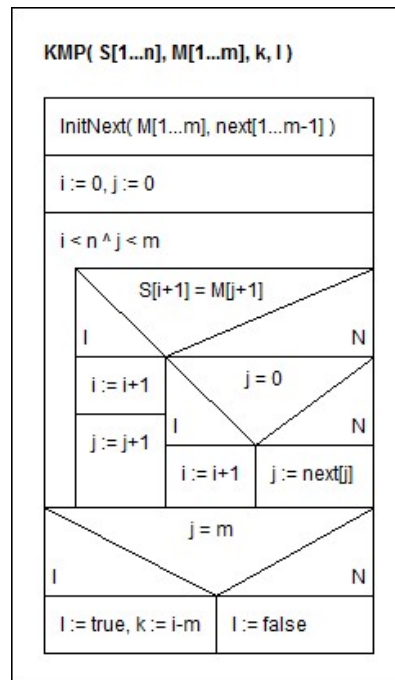


Figure 11: KMP algorithm

6.3 Boyer-Moore | Quick search algorithm

While the KMP algorithm decided on the shift based on the part before the place of breakdown, the QS decides based on the character after the pattern. So in case of a breakdown:

- If the character after the pattern is in the pattern, then we fit it to its first occurrence from the right. (Figure 12)

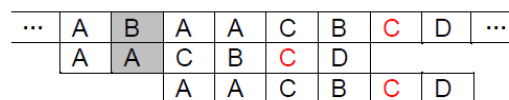


Figure 12: QS - shift if the character after the pattern is in the pattern

- If the character after the pattern is not in the pattern, then we fit the pattern after this character. (Figure 13)

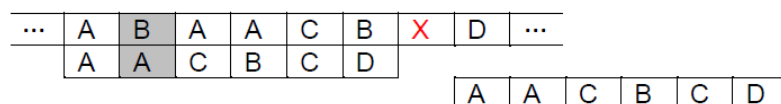
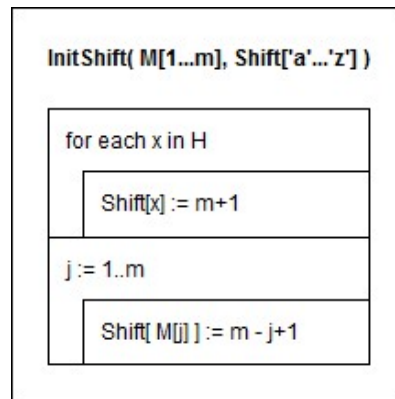


Figure 13: QS - shift if the character after the pattern is not in the pattern

The computation of the shift can again be aided with an array, but now, since it is not the pattern that is interesting, and we do not know exactly which characters appear in the text, we have to put the whole alphabet into the array (Figure 14)

Figure 14: QS - Constructing the array aiding the shift ($Shift['a'...'z']$)

The algorithm (see Figure 15):

- We run two indices k and j on the text and on the pattern, respectively.
- If the $(k+j)$ -th element of the text coincides with the j -th character of the pattern, then we advance j (since in the text we look at the $(k+j)$ -th element, so it is enough to increase j).
- If they do not coincide, then:
 - If the pattern is already at the end of the text ($k = n - m$), then we just increase k by one, which makes the loop condition false.
 - If we are not yet at the end of the text, we shift k by as much as is possible (this is determined by the preset $Shift$ array). And we reset j to 1.
- We go until we reach either the end of the pattern with j , or we shifted the pattern past the end of the text. In the former case we found a match, while in the latter we did not.

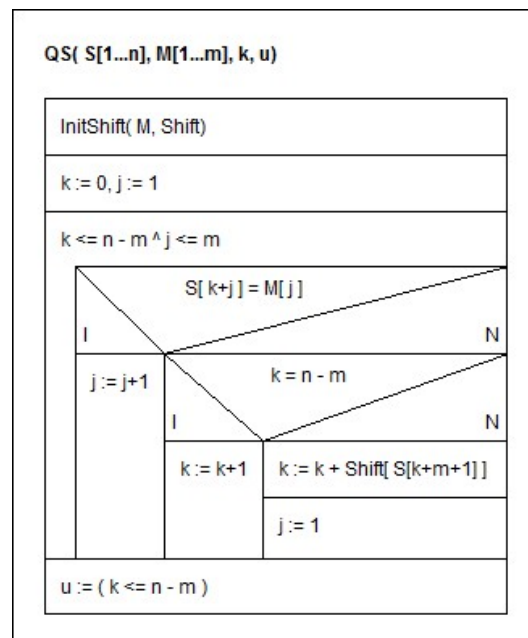


Figure 15: QS

6.4 Rabin-Karp algorithm

The essence of the Rabin-Karp algorithm is that we assign to every letter of the alphabet a digit, and perform the search by comparing numbers. It is clear that for this we will need a number system corresponding to the alphabet

size. From the text we always cut out parts equal to the length of the pattern, and compare these.

Example:

Pattern: BBAC $\rightarrow$ 1102

Text: DACABBAC $\rightarrow$ 30201102, from which we produce the following numbers: 3020, 0201, 2011, 0110, 1102

The slices seen above will be the s_i .

For the operation of the algorithm, however, we apply several small ideas:

1. We perform the conversion of the pattern into numbers with the help of Horner's method.

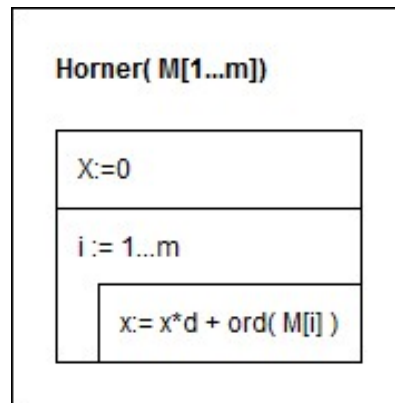


Figure 16: RK - Horner's method

The $ord()$ function returns the number corresponding to each letter. d is the base of the number system.

2. Producing the slices (s_i) of the text equal in length to the pattern:

We can compute s_0 with Horner's method. After this s_{i+1} is to be computed as follows:

$$s_{i+1} = (s_i - ord(S[i]) \cdot d^{m-1}) \cdot d + ord(S[i+1])$$

Explanation: from the beginning of s_i we cut off the first digit ($s_i - ord(S[i]) \cdot d^{m-1}$), then we shift the remainder by one place value (multiplication by d), finally into the last place value we write the digit corresponding to the next letter ($+ord(S[i+1])$)

Example:

With the text and pattern of the previous example ($d = 10$ -element alphabet and $m = 4$ long pattern):

$s_0 = 3020$, then: $s_{0+1} = s_1 = (3020 - ord(D) \cdot 10^3) \cdot 10 + ord(B) = (3020 - 3000) \cdot 10 + 1 = 0201$

3. The question may arise whether number systems of such high base do not cause a problem in the representation. The question is justified. Take the following lifelike example:

We represent our numbers on 4 bytes (2^{32}). Let the alphabet be of 32 elements ($d = 32$), the pattern 8 long ($m = 8$). Then the computation of d^{m-1} : $32^7 = (2^5)^7 = 2^{35}$, which is already not representable on 4 bytes.

To eliminate this, let us introduce a large prime p for which $d \cdot p$ is still representable. And let us compute the operations $\text{mod } p$. Then, of course, because of the congruence there will be a case when the algorithm shows a match when in reality there is none. This does not cause a problem, since in such a case, examining a character-by-character match, we can handle this problem. (The reverse case does not occur, so there will be no case when there is a character-by-character match but not a numeric one). [If p is sufficiently large, the phenomenon occurs very rarely.]

4. The $\text{mod } p$ computation also raises another problem. Namely, during the subtraction we can also get negative numbers.

For example: Let $p = 7$; then if $\text{ord}(S[i]) = 9$, then after the previous computation $s_i = 2\dots$, but from this, subtracting $\text{ord}(S[i]) \cdot d^{m-1} = 9 \cdot 10^3 = 9000$, we get a negative number.

As a solution we compute s_{i+1} in two steps:

$$s := (s_i + d \cdot p - \text{ord}(S[i]) \cdot d^{m-1}) \mod p$$

$$s_{i+1} := (s \cdot d + \text{ord}(S[i+1])) \mod p$$

Based on the above, the algorithm is the following (see Figure 17)

1. We compute d^{m-1} ($dm1$)
2. In one iteration we determine with Horner's method the number of the pattern (x) and s_0
3. We check whether they coincide
4. We keep computing the value of s_i until the pattern coincides with s_i , or the pattern reaches the end of the text.

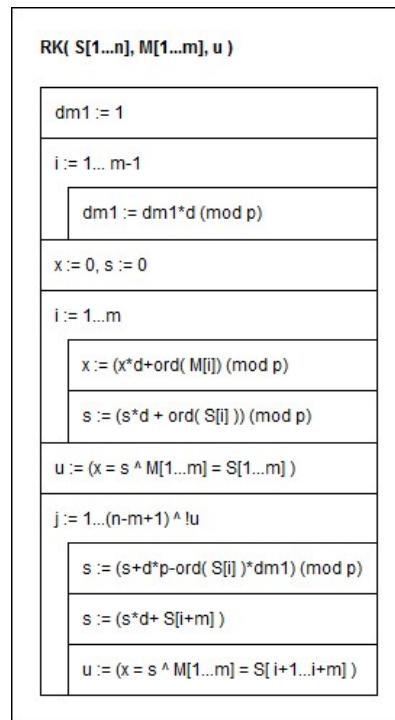


Figure 17: RK